

TP Integrador POO — UNQ-Shop

Guía para la defensa (speech + decisiones + dudas)

Sistema de e-commerce, **solo capa de dominio** en Java (sin GUI, sin persistencia, sin frameworks). Lo que se evalúa es la **correcta aplicación de patrones de diseño** y saber justificar cada decisión.

1. El pitch de 30 segundos

«UNQ-Shop es un sistema de venta online del que implementamos la capa de dominio. Se organiza en 7 módulos y cada uno resuelve un problema de diseño con un patrón distinto. El corazón es la clase **Pedido**, que actúa como punto de encuentro: es el **contexto** del State (su ciclo de vida), el **sujeto** del Observer (las notificaciones), y el **cliente** de las estrategias de envío y del método de pago. Alrededor tenemos el catálogo (Composite), la búsqueda (Specification) y los reportes (Visitor).»

Mapa de patrones (memorizar esta tabla)

Módulo	Patrón	Idea en una línea
2.1 Catálogo	Composite	Tratar producto y paquete igual; precio recursivo
2.2 Pedido	State	El comportamiento cambia según el estado
2.3 Envío	Strategy	Algoritmo de costo intercambiable
2.4 Pago	Template Method	Pasos fijos en orden, contenido variable
2.5 Notificaciones	Observer	Avisar sin conocer a los subsistemas
2.6 Búsqueda	Composite + Specification	Criterios combinables y anidables
2.8 Reportes	Visitor	Separar los datos del formato de salida

2. Cómo defender cada módulo

2.1 Catálogo → Composite

Qué resuelve: el catálogo tiene productos individuales y paquetes; un paquete puede contener productos *u otros paquetes*, y el precio se calcula de forma recursiva.

Por qué Composite: porque es una estructura de árbol (todo-parte) donde las hojas y los nodos deben tratarse igual. Definimos la clase abstracta **ItemCatalogo** (el *Component*); **Producto** es la

hoja (*Leaf*) y **Paquete** es el compuesto (*Composite*), que guarda una lista de `ItemCatalogo`. Como un hijo puede ser otro `Paquete`, la recursión sale natural.

Alternativa frágil que evitamos: andar preguntando `if (item instanceof Paquete)` por todos lados y hacer la recursión a mano; eso se rompe al anidar y viola `Open/Closed`.

Dato extra — atributos dinámicos: el enunciado pedía agregar atributos que NO son campos de la clase (`Alto`, `Ancho`, `Peso`). Lo resolvimos con un **`Map<String, Object>`** (patrón *Properties*): así se agregan atributos sin tocar ni recompilar la clase. La validación (`esValido`) chequea que nombre y SKU estén, y que los dinámicos tengan valor asignado.

Frase para la defensa: «Usé `Composite` porque el cliente que navega el catálogo no necesita saber si consulta un producto o un paquete: ambos responden al mismo contrato `getPrecioFinal()`, y el paquete simplemente delega en sus hijos.»

Roles GoF: `ItemCatalogo` = `Component` · `Producto` = `Leaf` · `Paquete` = `Composite`.

2.2 Ciclo de vida del pedido → State

Qué resuelve: un pedido cambia de comportamiento según su estado (`Borrador`, `Confirmado`, `EnPreparacion`, `Enviado`, `Entregado`, `Cancelado`). Cada estado permite ciertas operaciones; una operación inválida lanza una excepción de dominio propia (**`PedidoException`**).

Por qué State: porque la misma operación (confirmar, cancelar, enviar...) hace algo distinto —o es inválida— según el estado. Cada estado es una clase que sabe sus transiciones válidas. El **Pedido** es el contexto: delega en su estado actual y este decide qué hacer y a qué estado pasar.

Detalle clave — cómo evitamos repetir código: creamos **`EstadoPedidoBase`**, una clase abstracta que por defecto lanza `PedidoException` en toda operación. Cada estado concreto solo redefine las operaciones que SÍ son válidas. Así no repetimos los «métodos vacíos / inválidos» en cada clase.

Detalle fino — reembolsos asimétricos (te lo pueden preguntar): cancelar desde **`EnPreparacion`** repone el stock y reembolsa producto + envío; cancelar desde **`Enviado`** reembolsa **solo el producto** (no repone stock ni devuelve el envío). Eso vive en el `cancelar()` de cada estado.

Alternativa frágil que evitamos: un `switch(estado)` gigante repetido dentro de cada método del pedido; agregar un estado obligaría a tocar todos los métodos.

Frase para la defensa: «Cada estado es una clase con sus transiciones válidas. Las inválidas las hereda de una superclase abstracta que lanza la excepción, así no duplico código. El pedido no sabe la lógica: la delega en su estado.»

Roles GoF: `Pedido` = `Context` · `EstadoPedido` = `State` · las 6 clases = `ConcreteState`.

2.3 Métodos de envío → Strategy

Qué resuelve: hay tres formas de calcular el costo del envío (estándar, express, retiro en sucursal), intercambiables.

Por qué Strategy: porque lo que varía es **un algoritmo** (el cálculo del costo). Definimos la interfaz **EstrategiaEnvio** con `costoEnvio(pedido)` y `diasEstimados(pedido)`; cada estrategia concreta encapsula su cálculo y sus datos. El Pedido tiene una EstrategiaEnvio y le delega.

Cómo funciona cada una: Estándar usa peso + dirección (delega en la librería externa **CorreoArgentina**); Express cobra un % del total + un cargo base; Retiro en sucursal cuesta 0 y calcula los días según si hay stock en esa sucursal.

Frase para la defensa: «Cada forma de envío es una estrategia intercambiable detrás de la misma interfaz. El Pedido pide el costo sin saber cómo se calcula; puedo agregar un envío nuevo sin tocar el Pedido.»

Roles GoF: EstrategiaEnvio = Strategy · las 3 clases = ConcreteStrategy · Pedido = Context.

2.4 Métodos de pago → Template Method

Qué resuelve: pagar siempre sigue los mismos pasos en el mismo orden (validar → reservar → ejecutar → notificar), pero cada medio los implementa distinto.

Por qué Template Method y NO Strategy: porque lo invariante es el **orden de los pasos**. En la clase abstracta **MetodoDePago** el método `pagar()` es **final** y llama en orden a los cuatro pasos; los tres primeros son abstractos (obligatorios) y `notificarResultado()` es un **hook** con una implementación por defecto que las subclases pueden personalizar. Subclases: Tarjeta, Transferencia, Billetera.

Las interfaces API: el enunciado pedía **definir** (no implementar) una interfaz-API por medio de pago para hablar con los servicios externos (banco, billetera). Por eso hay `TarjetaDeCreditoAPI`, `TransferenciaBancariaAPI`, `BilleteraVirtualAPI`: se usan pero no se implementan.

Frase para la defensa: «El esqueleto del algoritmo —el orden de los pasos— vive en la clase abstracta y es final para que nadie lo altere. Cada medio de pago solo rellena los pasos. Notificar es un hook opcional.»

Roles GoF: MetodoDePago = AbstractClass (define el Template Method + el hook) · cada medio = ConcreteClass.

2.5 Notificaciones → Observer

Qué resuelve: cuando el pedido cambia de estado, varios subsistemas reaccionan de forma independiente (mandar mail, generar factura, dar cupón de fidelización), y el pedido no debe conocerlos. Deben poder sumarse subsistemas nuevos.

Por qué Observer: porque es una relación uno-a-muchos con acoplamiento débil. El **Pedido** (sujeto) tiene un **Notificador** con una lista de **Suscriptores**; cuando cambia de estado (`setEstado`) los notifica. Los suscriptores concretos deciden si actúan.

Cómo deciden los suscriptores: la interfaz `EstadoPedido` tiene métodos por defecto `debeNotificarPorEmail()`, `debeGenerarFactura()`, `debeFidelizar()` que devuelven `false`; cada estado los redefine. Así el **SuscriptorCorreo** solo manda mail si el estado dice que corresponde, el **Facturador** factura al entregar, y el **Fidelizador** da el cupón 5% al cancelar.

Alternativa frágil que evitamos: que el Pedido llame directo a mail, factura y fidelización; eso lo acopla y obliga a editarlo por cada subsistema nuevo. Con Observer, un subsistema nuevo solo se suscribe.

Frase para la defensa: «El pedido solo grita 'cambié de estado' y no sabe quién escucha. Cada suscriptor decide si le interesa ese estado. Sumar uno nuevo no toca el Pedido.»

Roles GoF: Pedido/Notificador = Subject · Suscriptor = Observer · Correo/Facturador/Fidelizador = ConcreteObserver.

2.6 Búsqueda → Composite + Specification

Qué resuelve: buscar ítems con criterios simples (nombre, precio máximo, categoría, disponibilidad) combinables con AND / OR / NOT, anidables sin límite.

Por qué Specification + Composite: cada criterio es un objeto que sabe responder `cumpleConCriterio(item)` (eso es Specification). Los operadores **CriterioAnd**, **CriterioOr** y **CriterioNot** también son criterios que contienen otros criterios (eso es Composite otra vez). Así se anidan libremente. El **Buscador** solo filtra el catálogo aplicando el criterio; no decide cómo evaluar.

Frase para la defensa: «Un criterio sabe si un ítem lo cumple. Los operadores lógicos también son criterios que envuelven otros, así que puedo armar `AND(Categoria, NOT(PrecioMax))` con la profundidad que quiera. El buscador es tonto: solo aplica el criterio que le pasan.»

Roles GoF: Criterio = Component · los simples = Leaf · And/Or/Not = Composite.

2.8 Reportes → Visitor

Qué resuelve: un reporte se exporta en varios formatos (texto, CSV, HTML) y el núcleo de los datos debe ser independiente del formato.

Por qué Visitor: porque queremos separar «qué se reporta» de «cómo se presenta». La interfaz **ReporteVisitor** tiene `visitarProducto` y `visitarPaquete`; hay un visitante concreto por formato: **ReporteTexto**, **ReporteCSV**, **ReportHTML**. Agregar un formato nuevo = un visitante nuevo, sin tocar los datos.

Alternativa frágil que evitamos: meter `toTexto()` / `toCsv()` / `toHtml()` dentro de `Producto` y `Paquete`; eso mezcla datos con formato y obliga a editar todo por cada formato nuevo.

Frase para la defensa: «Los datos (producto/paquete) no saben de formatos. El formato es un visitante: para sumar HTML solo agregué una clase, sin tocar el catálogo.»

Roles GoF: `ReporteVisitor = Visitor · Texto/CSV/HTML = ConcreteVisitor · Producto/Paquete = Element`.

3. Preguntas trampa y dudas que surgieron

¿State vs Strategy? (la más típica)

Estructuralmente son casi idénticos: un objeto delega en otro. Cambia la **intención**. **Strategy** = elegir un *algoritmo* intercambiable (el envío). **State** = el objeto *cambia de estado por sí mismo* y eso altera su comportamiento (el pedido). Y una diferencia concreta: en State, un estado **instancia a su estado hermano** para hacer la transición; en Strategy no.

¿Template Method vs Strategy? (pago vs envío)

En el pago lo que es fijo es el **orden de los pasos**, por eso usamos herencia (Template Method): el esqueleto vive en la clase abstracta. En el envío varía el algoritmo entero, por eso usamos delegación (Strategy). Regla: si comparten la secuencia y solo cambian los pasos → Template Method; si es intercambiable el algoritmo completo → Strategy.

¿Por qué Composite aparece dos veces? (catálogo y búsqueda)

Son dos árboles distintos: uno de *productos* (paquetes que contienen paquetes) y otro de *criterios* (AND/OR/NOT que contienen criterios). El mismo patrón resuelve dos problemas con estructura de árbol.

¿Por qué los atributos dinámicos son un Map y no campos de la clase?

Porque el enunciado pedía agregar atributos «que no sean atributos de una clase». Con un `Map<String, Object>` el modelo es escalable: sumo Alto/Ancho/Peso a un producto puntual sin modificar ni recompilar la clase `Producto`.

¿Precio base vs precio final del paquete?

Decidimos: `getPrecioBase()` = la suma de los precios base de los ítems (sin descuento); `getPreciofinal()` = la suma de los precios finales $\times (1 - \text{descuento del paquete})$. Con el ejemplo del enunciado: $(8000+1500+800) = 10300$ de base, y $\times 0,85 = 8755$ final. Es recursivo, así que un paquete dentro de otro aporta su final ya con su propio descuento.

¿Por qué las APIs de pago son interfaces sin implementar?

Porque representan servicios externos (banco, billetera) que no son parte de nuestro dominio. El enunciado pedía definir las como contrato (API) para poder invocarlas, no implementarlas.

¿Cómo testearon si el 95% de cobertura?

Con pocos **tests de escenario grandes** que recorren un caso de uso completo y relacionan todos

los módulos (ej. «compra feliz»: arma catálogo → confirma → cobra → descuenta stock → avanza estados → notifica), más algunos tests puntuales para cerrar ramas sueltas. Llegamos a ~99% de líneas. La idea: demostrar que las piezas encajan entre sí, no probar cada método aislado.

Duda honesta que puede aparecer — el Visitor sin `acceptar()`:

En un Visitor «de libro» cada elemento tiene un método `acceptar(visitor)` que hace el doble despacho. En nuestra implementación el reporte llama directo a `visitarProducto/visitarPaquete`. Funciona, pero si lo preguntan: la versión más canonónica agregaría `acceptar()` en `ItemCatalogo` para el doble despacho.

Duda honesta — el bug del notificador (si sale el tema de testing):

Al escribir los tests detectamos que el `notificador` del Pedido no se inicializaba, así que cualquier cambio de estado tiraba `NullPointerException`. Lo arreglamos inicializándolo. Es un buen ejemplo de **para qué sirven los tests**: encontrar errores antes de la entrega.

4. Speech de cierre (para arrancar la defensa)

«Nuestro TP modela un e-commerce en su capa de dominio. La decisión central fue identificar, en cada módulo, el problema de diseño y el patrón que lo resuelve evitando la alternativa frágil. El catálogo es un **Composite** porque productos y paquetes se tratan igual y el precio es recursivo. El pedido usa **State** porque su comportamiento cambia según la etapa. El envío es **Strategy** (algoritmo intercambiable) y el pago es **Template Method** (pasos fijos en orden). Las notificaciones son **Observer** para desacoplar al pedido de los subsistemas. La búsqueda combina **Composite y Specification** para criterios anidables, y los reportes usan **Visitor** para separar los datos del formato. Todo está cubierto con tests de escenario que demuestran que los módulos funcionan integrados.»

Consejo: en la entrevista, para cada patrón decí siempre las 3 cosas: (1) qué problema resuelve, (2) cómo lo implementaste, (3) qué alternativa frágil evitaste. Con eso demostrás que entendés el porqué, no solo el cómo.